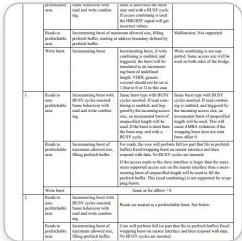


| What                            | Old TB              | Fixed TB                                                                      |
|---------------------------------|---------------------|-------------------------------------------------------------------------------|
| SRAM model                      | Always drives rdata | Only drives rdata when <code>ce1=0 AND oen=0 AND wen=1</code>                 |
| Code structure                  | One big process     | Separate AHB master model + stimulus + <code>do_ahb_transfer</code> procedure |
| <code>ahb_done</code> handshake | None                | Stimulus waits for <code>ahb_done=1</code> before next test                   |



The first code with just the sram model and apb slave model all worked perfectly after some debugging on the design. Now I want you to create me a testbench that has a AHB Master model as well and it should test all the 8 burst modes along with the read and write(from the first code u gave me). (Also Mention to me what other tests u think might be useful and i will let u know whether i need them. I have given a table photo which i think might be helpful..but if it is not don't use it . Give me the code in chat

Great that the first TB worked! The table is very helpful — it's the GRLIB AHB bridge burst handling table. Let me use it.

Other Tests That Might Be Useful (Tell me which you want)

| # | Test                            | Why Useful                                   |
|---|---------------------------------|----------------------------------------------|
| A | Error response (HRESP=ERROR)    | Check bridge handles slave errors gracefully |
| B | Back-to-back bursts             | No gap between two INCR4 bursts              |
| C | Locked transfer (hlock=1)       | Verify bus locking works                     |
| D | Mixed proc + AHB simultaneously | Both bridges active at same time             |

| # | Test                         | Why Useful                                |
|---|------------------------------|-------------------------------------------|
| E | <b>Reset mid-transaction</b> | Assert rst during a burst, check recovery |
| F | <b>IRQ assertion check</b>   | Verify irq line toggles correctly         |
| G | <b>BUSY cycle insertion</b>  | Master inserts BUSY within a burst        |

## The Full Testbench With All 8 Burst Types

```

vhdl

-- =====
-- tb_bridge_top_burst.vhd
-- Testbench for DUT: eth_module
-- Tests: All 8 AHB burst modes + proc read/write
-- Simulator: VCS
-- =====

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.bridge_pkg.all;

entity tb_bridge_top is
end entity tb_bridge_top;

architecture sim of tb_bridge_top is

    -- =====
    -- Constants
    -- =====

    constant ABITS      : integer := 28;
    constant CLK_PERIOD : time    := 10 ns;

    -- AHB HTRANS encoding
    constant IDLE      : std_logic_vector(1 downto 0) := "00";
    constant BUSY      : std_logic_vector(1 downto 0) := "01";
    constant NONSEQ    : std_logic_vector(1 downto 0) := "10";
    constant SEQ       : std_logic_vector(1 downto 0) := "11";

    -- AHB HBURST encoding
    constant SINGLE    : std_logic_vector(2 downto 0) := "000";
    constant INCR      : std_logic_vector(2 downto 0) := "001";
    constant WRAP4     : std_logic_vector(2 downto 0) := "010";
    constant INCR4     : std_logic_vector(2 downto 0) := "011";

```

```

constant WRAP8      : std_logic_vector(2 downto 0) := "100";
constant INCR8      : std_logic_vector(2 downto 0) := "101";
constant WRAP16     : std_logic_vector(2 downto 0) := "110";
constant INCR16     : std_logic_vector(2 downto 0) := "111";

-- AHB HSIZE encoding
constant SIZE_BYTE   : std_logic_vector(2 downto 0) := "000";
constant SIZE_HWORD  : std_logic_vector(2 downto 0) := "001";
constant SIZE_WORD   : std_logic_vector(2 downto 0) := "010"; -- 32-bit

-- =====
-- DUT port signals
-- =====

signal clk   : std_logic := '0';
signal rst   : std_logic := '0';

signal hbusreq : std_ulogic           := '0';
signal hlock   : std_ulogic           := '0';
signal htrans  : std_logic_vector(1 downto 0) := IDLE;
signal haddr   : std_logic_vector(31 downto 0) := (others => '0');
signal hwrite  : std_ulogic           := '0';
signal hsize   : std_logic_vector(2 downto 0) := SIZE_WORD;
signal hburst  : std_logic_vector(2 downto 0) := SINGLE;
signal hprot   : std_logic_vector(3 downto 0) := "0011";
signal hwdata  : std_logic_vector(31 downto 0) := (others => '0');

signal hgrant  : std_ulogic;
signal hready  : std_ulogic;
signal hresp   : std_logic_vector(1 downto 0);
signal hrdata  : std_logic_vector(31 downto 0);

signal prdata  : std_logic_vector(31 downto 0) := (others => '0');
signal psel    : std_ulogic;
signal penable : std_ulogic;
signal paddr   : std_logic_vector(31 downto 0);
signal pwrite  : std_ulogic;
signal pwdata  : std_logic_vector(31 downto 0);

signal sram_rdata : std_logic_vector(31 downto 0) := (others => '0');
signal sram_addr  : std_logic_vector(ABITS-1 downto 0);
signal sram_wdata : std_logic_vector(31 downto 0);
signal sram_ce1   : std_logic;
signal sram_wen   : std_ulogic;
signal sram_oen   : std_ulogic;

signal proc_wdata : std_logic_vector(31 downto 0) := (others => '0');

```

```

signal proc_addr   : std_logic_vector(ABITS-1 downto 0) := (others => '0');
signal proc_ce1    : std_logic := '1';
signal proc_wen    : std_logic := '1';
signal proc_oen    : std_logic := '1';
signal proc_rdata  : std_logic_vector(31 downto 0);

signal testrst : std_ulogic;
signal testen  : std_ulogic;
signal testoen : std_ulogic;
signal irq     : std_logic;

-- =====
-- AHB Master model handshake signals
-- =====
-- Stimulus fills these, then pulses ahb_start
-- AHB model reads them and pulses ahb_done when complete
signal ahb_start      : std_logic := '0';
signal ahb_done       : std_logic := '0';

-- Transaction descriptor
signal ahb_req_addr    : std_logic_vector(31 downto 0) := (others => '0');
signal ahb_req_wdata0  : std_logic_vector(31 downto 0) := (others => '0');
signal ahb_req_wdata1  : std_logic_vector(31 downto 0) := (others => '0');
signal ahb_req_wdata2  : std_logic_vector(31 downto 0) := (others => '0');
signal ahb_req_wdata3  : std_logic_vector(31 downto 0) := (others => '0');
signal ahb_req_wdata4  : std_logic_vector(31 downto 0) := (others => '0');
signal ahb_req_wdata5  : std_logic_vector(31 downto 0) := (others => '0');
signal ahb_req_wdata6  : std_logic_vector(31 downto 0) := (others => '0');
signal ahb_req_wdata7  : std_logic_vector(31 downto 0) := (others => '0');
signal ahb_req_write    : std_ulogic := '0';
signal ahb_req_burst    : std_logic_vector(2 downto 0) := SINGLE;
signal ahb_req_size     : std_logic_vector(2 downto 0) := SIZE_WORD;
signal ahb_busy_beat    : integer := 99; -- beat index after which BUSY inserted
                                     -- 99 = no BUSY inserted

-- =====
-- Component Declaration
-- =====
component eth_module is
  generic (
    constant SRAMBANKS : integer := 4;
    constant TACC      : integer := 10;
    constant ABITS     : integer := 28;
    constant sram_file : string  := "sram.srec"
  );
  port (

```

```
    clk, rst      : in  std_logic;
    hbusreq       : in  std_ulogic;
    hlock         : in  std_ulogic;
    htrans        : in  std_logic_vector(1 downto 0);
    haddr         : in  std_logic_vector(31 downto 0);
    hwrite        : in  std_ulogic;
    hsize         : in  std_logic_vector(2 downto 0);
    hburst        : in  std_logic_vector(2 downto 0);
    hprot         : in  std_logic_vector(3 downto 0);
    hwdata        : in  std_logic_vector(31 downto 0);
    hgrant        : out std_ulogic;
    hready        : out std_ulogic;
    hresp         : out std_logic_vector(1 downto 0);
    hrdata        : out std_logic_vector(31 downto 0);
    prdata        : in  std_logic_vector(31 downto 0);
    psel          : out std_ulogic;
    penable       : out std_ulogic;
    paddr         : out std_logic_vector(31 downto 0);
    pwrite        : out std_ulogic;
    pwdata        : out std_logic_vector(31 downto 0);
    sram_rdata     : in  std_logic_vector(31 downto 0);
    sram_addr     : out std_logic_vector(ABITS-1 downto 0);
    sram_wdata     : out std_logic_vector(31 downto 0);
    sram_ce1      : out std_logic;
    sram_wen      : out std_ulogic;
    sram_oen      : out std_ulogic;
    proc_wdata     : in  std_logic_vector(31 downto 0);
    proc_addr     : in  std_logic_vector(ABITS-1 downto 0);
    proc_ce1      : in  std_logic;
    proc_wen      : in  std_logic;
    proc_oen      : in  std_logic;
    proc_rdata     : out std_logic_vector(31 downto 0);
    testrst       : out std_ulogic;
    testen        : out std_ulogic;
    testoen       : out std_ulogic;
    irq           : out std_logic
);
end component;

begin

-- =====
-- DUT Instantiation
-- =====

DUT : eth_module
    generic map (
```

```

SRAMBANKS => 4,
TACC       => 10,
ABITS      => 28,
sram_file  => "sram.srec"
)
port map (
    clk      => clk,      rst      => rst,
    hbusreq   => hbusreq,  hlock   => hlock,
    htrans    => htrans,  haddr   => haddr,
    hwrite    => hwrite,  hsize   => hsize,
    hburst    => hburst,  hprot   => hprot,
    hwddata   => hwddata, hgrant   => hgrant,
    hready    => hready,  hresp   => hresp,
    hrdata    => hrdata,  prdata   => prdata,
    psel      => psel,    penable  => penable,
    paddr     => paddr,    pwrite   => pwrite,
    pwdata    => pwdata,   sram_rdata => sram_rdata,
    sram_addr  => sram_addr, sram_wdata => sram_wdata,
    sram_ce1   => sram_ce1, sram_wen  => sram_wen,
    sram_oen   => sram_oen, proc_wdata => proc_wdata,
    proc_addr  => proc_addr, proc_ce1  => proc_ce1,
    proc_wen   => proc_wen, proc_oen   => proc_oen,
    proc_rdata => proc_rdata, testrst  => testrst,
    testen    => testen,   testoen   => testoen,
    irq       => irq
);

-- =====
-- Clock Generation: 100 MHz
-- =====
clk_gen : process
begin
    clk <= '0'; wait for CLK_PERIOD / 2;
    clk <= '1'; wait for CLK_PERIOD / 2;
end process;

-- =====
-- SRAM Model
-- READ:  ce1=0, oen=0, wen=1 → return addr-tagged data
-- WRITE: ce1=0, wen=0          → absorb data (no storage)
-- The addr tag lets you verify correct address on waveform
-- =====
sram_model : process(sram_ce1, sram_oen, sram_wen, sram_addr)
begin
    if sram_ce1 = '0' and sram_oen = '0' and sram_wen = '1' then
        -- Return lower 8 bits of address as top byte

```

```

-- e.g. addr=0x040 → rdata=0x40_AB_CD_EF
sram_rdata <= sram_addr(7 downto 0) & x"ABCDEF";
else
  sram_rdata <= (others => '0');
end if;
end process;

-- =====
-- APB Slave Model
-- SETUP cycle: psel=1, penable=0
-- ENABLE cycle: psel=1, penable=1 → respond here
-- READ: return addr-tagged data e.g. 0x10FACADE
-- WRITE: acknowledge silently
-- =====
apb_slave_model : process(clk)
begin
  if rising_edge(clk) then
    if psel = '1' and penable = '1' then
      if pwrite = '0' then
        prdata <= paddr(7 downto 0) & x"FACADE";
      else
        prdata <= (others => '0');
      end if;
    else
      prdata <= (others => '0');
    end if;
  end if;
end process;

-- =====
-- AHB MASTER MODEL
-- Implements full AHB-Lite master protocol:
--
-- SINGLE:
--   [req bus] → [wait grant+ready] →
--   [addr phase: NONSEQ] → [wait hready] →
--   [data phase: hwddata/sample hrdata] → [IDLE]
--
-- INCR (undefined length):
--   [req bus] → [wait grant+ready] →
--   [beat0: NONSEQ addr] → [wait hready] →
--   [beat1..N-1: SEQ addr + prev beat data] → [wait hready each] →
--   [last data phase] → [IDLE]
--   Note: N beats determined by burst type
--   INCR = 1 beat (we use 4 to show undefined length)
--

```

```

-- INCR4/INCR8/INCR16:
--     Fixed beat count, address increments by hsize each beat
--
-- WRAP4/WRAP8/WRAP16:
--     Fixed beat count, address wraps at boundary
--     WRAP4  wraps at 4-beat boundary (16  bytes for word)
--     WRAP8  wraps at 8-beat boundary (32  bytes for word)
--     WRAP16 wraps at 16-beat boundary (64  bytes for word)
--
-- BUSY cycle:
--     Master inserts BUSY (htrans=01) between beats to stall
--     Bridge must hold data and wait
-- =====
ahb_master_model : process

-- Number of beats for each burst type
function beat_count(burst : std_logic_vector(2 downto 0))
    return integer is
begin
    case burst is
        when SINGLE => return 1;
        when INCR   => return 4;    -- undefined: we do 4 beats
        when WRAP4  => return 4;
        when INCR4  => return 4;
        when WRAP8  => return 8;
        when INCR8  => return 8;
        when WRAP16 => return 16;
        when INCR16 => return 16;
        when others => return 1;
    end case;
end function;

-- Address increment per beat (bytes), based on hsize
function addr_increment(size : std_logic_vector(2 downto 0))
    return integer is
begin
    case size is
        when SIZE_BYTE   => return 1;
        when SIZE_HWORD  => return 2;
        when SIZE_WORD   => return 4;
        when others      => return 4;
    end case;
end function;

-- Wrap mask: address bits that wrap
-- WRAP4  word = wraps within 16-byte boundary → mask=0xF

```

```
-- WRAP8  word = wraps within 32-byte boundary → mask=0x1F
-- WRAP16 word = wraps within 64-byte boundary → mask=0x3F
function wrap_mask(burst : std_logic_vector(2 downto 0);
                  size   : std_logic_vector(2 downto 0))
    return unsigned is
        variable beats : integer;
        variable inc   : integer;
begin
    beats := beat_count(burst);
    inc   := addr_increment(size);
    return to_unsigned(beats * inc - 1, 32);
end function;

-- Next address calculation (handles wrapping)
function next_addr(
    current : std_logic_vector(31 downto 0);
    start   : std_logic_vector(31 downto 0);
    burst   : std_logic_vector(2 downto 0);
    size    : std_logic_vector(2 downto 0))
    return std_logic_vector is
        variable inc      : integer;
        variable mask     : unsigned(31 downto 0);
        variable cur_u    : unsigned(31 downto 0);
        variable start_u  : unsigned(31 downto 0);
        variable next_u   : unsigned(31 downto 0);
begin
    inc      := addr_increment(size);
    cur_u    := unsigned(current);
    start_u  := unsigned(start);

    case burst is
        when WRAP4 | WRAP8 | WRAP16 =>
            -- Wrapping: only lower bits change, upper bits fixed
            mask := wrap_mask(burst, size);
            next_u := (cur_u and not mask) or
                      ((cur_u + inc) and mask);
        when others =>
            -- Incrementing: just add
            next_u := cur_u + inc;
    end case;

    return std_logic_vector(next_u);
end function;

-- Get wdata for beat index
function get_wdata(beat : integer) return std_logic_vector is
```

```

begin
  case beat is
    when 0 => return ahb_req_wdata0;
    when 1 => return ahb_req_wdata1;
    when 2 => return ahb_req_wdata2;
    when 3 => return ahb_req_wdata3;
    when 4 => return ahb_req_wdata4;
    when 5 => return ahb_req_wdata5;
    when 6 => return ahb_req_wdata6;
    when 7 => return ahb_req_wdata7;
    when others => return x"00000000";
  end case;
end function;

variable n_beats      : integer;
variable cur_addr     : std_logic_vector(31 downto 0);
variable start_addr   : std_logic_vector(31 downto 0);

begin
  -- Default idle state
  hbusreq <= '0';
  hlock   <= '0';
  htrans  <= IDLE;
  haddr   <= (others => '0');
  hwrite  <= '0';
  hsize   <= SIZE_WORD;
  hburst  <= SINGLE;
  hprot   <= "0011";
  hwdata  <= (others => '0');
  ahb_done <= '0';

  -- Wait for stimulus to trigger
  wait until ahb_start = '1';

  n_beats      := beat_count(ahb_req_burst);
  start_addr   := ahb_req_addr;
  cur_addr     := ahb_req_addr;

  -- — Step 1: Request bus —————
  hbusreq <= '1';
  hsize   <= ahb_req_size;
  hburst  <= ahb_req_burst;
  wait until rising_edge(clk);

  -- — Step 2: Wait for grant AND ready —————
  while not (hgrant = '1' and hready = '1') loop

```

```
    wait until rising_edge(clk);
end loop;

-- — Step 3: Address phase for beat 0 (NONSEQ) —————
htrans <= NONSEQ;
haddr  <= cur_addr;
hwrite <= ahb_req_write;

-- — Step 4: Remaining beats —————
-- AHB pipeline: address of beat N is presented while
-- data of beat N-1 is being transferred
for beat in 0 to n_beats - 1 loop

    -- Wait for current address phase to be accepted
    wait until rising_edge(clk) and hready = '1';

    -- Insert BUSY cycle if requested at this beat
    if beat = ahb_busy_beat then
        htrans <= BUSY;
        haddr  <= cur_addr;    -- Hold address during BUSY
        -- Hold BUSY for 2 cycles
        wait until rising_edge(clk) and hready = '1';
        wait until rising_edge(clk) and hready = '1';
    end if;

    -- Data phase of current beat / Address phase of next beat
    if ahb_req_write = '1' then
        hwddata <= get_wdata(beat);    -- Data for current beat
    end if;

    -- Advance address for next beat
    cur_addr := next_addr(cur_addr, start_addr,
                          ahb_req_burst, ahb_req_size);

    if beat < n_beats - 1 then
        -- More beats: SEQ transfer
        htrans <= SEQ;
        haddr  <= cur_addr;    -- Next beat address
    else
        -- Last beat: go IDLE
        htrans  <= IDLE;
        hbusreq <= '0';
        haddr   <= (others => '0');
    end if;

end loop;
```

```

-- Wait for last data phase to complete
wait until rising_edge(clk) and hready = '1';

hwrite    <= '0';
hwdatal   <= (others => '0');

-- — Step 5: Signal done —————
ahb_done <= '1';
wait until rising_edge(clk);
ahb_done <= '0';

wait until ahb_start = '0';

end process ahb_master_model;

-- =====
-- STIMULUS PROCESS
-- =====
stimulus : process

-- — Trigger AHB single transfer —————
procedure do_ahb_single (
    addr  : std_logic_vector(31 downto 0);
    wdata : std_logic_vector(31 downto 0);
    wr     : std_ulogic
) is
begin
    ahb_req_addr    <= addr;
    ahb_req_wdata0 <= wdata;
    ahb_req_write    <= wr;
    ahb_req_burst    <= SINGLE;
    ahb_req_size     <= SIZE_WORD;
    ahb_busy_beat    <= 99;          -- no BUSY
    ahb_start        <= '1';
    wait until rising_edge(clk) and ahb_done = '1';
    ahb_start <= '0';
    wait for 3 * CLK_PERIOD;
end procedure;

-- — Trigger AHB burst (4/8/16 beats) —————
procedure do_ahb_burst (
    addr  : std_logic_vector(31 downto 0);
    burst : std_logic_vector(2 downto 0);
    wr     : std_ulogic;
    busy   : integer := 99          -- insert BUSY after this beat

```

```
) is
begin
    ahb_req_addr    <= addr;
    -- Write data per beat – address-tagged so easy to verify
    ahb_req_wdata0 <= x"AA000000";
    ahb_req_wdata1 <= x"BB000001";
    ahb_req_wdata2 <= x"CC000002";
    ahb_req_wdata3 <= x"DD000003";
    ahb_req_wdata4 <= x"EE000004";
    ahb_req_wdata5 <= x"FF000005";
    ahb_req_wdata6 <= x"A1000006";
    ahb_req_wdata7 <= x"B2000007";
    ahb_req_write   <= wr;
    ahb_req_burst    <= burst;
    ahb_req_size     <= SIZE_WORD;
    ahb_busy_beat    <= busy;
    ahb_start        <= '1';
    wait until rising_edge(clk) and ahb_done = '1';
    ahb_start <= '0';
    wait for 3 * CLK_PERIOD;
end procedure;
```

```
-- — Processor write —————
procedure do_proc_write (
    addr  : std_logic_vector(ABITS-1 downto 0);
    wdata : std_logic_vector(31 downto 0)
) is
begin
    wait until rising_edge(clk);
    proc_addr <= addr;
    proc_wdata <= wdata;
    proc_ce1   <= '0';
    proc_wen   <= '0';
    proc_oen   <= '1';
    wait for 4 * CLK_PERIOD;
    proc_ce1   <= '1';
    proc_wen   <= '1';
    wait for 2 * CLK_PERIOD;
end procedure;
```

```
-- — Processor read —————
procedure do_proc_read (
    addr : std_logic_vector(ABITS-1 downto 0)
) is
begin
    wait until rising_edge(clk);
```

```

    proc_addr  <= addr;
    proc_wdata <= (others => '0');
    proc_ce1   <= '0';
    proc_wen   <= '1';
    proc_oen   <= '0';
    wait for 4 * CLK_PERIOD;
    proc_ce1   <= '1';
    proc_oen   <= '1';
    wait for 2 * CLK_PERIOD;
end procedure;

begin

-- --- Reset -----
rst      <= '0';
ahb_start <= '0';
wait for 5 * CLK_PERIOD;
rst <= '1';
wait for 5 * CLK_PERIOD;
report "=== RESET RELEASED ===" severity note;

-- =====
-- PROCESSOR TESTS (APB Bridge)
-- =====

-- Test P1: Processor single WRITE
-- Drive: proc_addr=0x10, proc_wdata=0xDEADBEEF
-- See:   psel=1, penable=1, paddr=0x10, pdata=0xDEADBEEF
report "=== P1: Processor WRITE ===" severity note;
do_proc_write(
    addr => std_logic_vector(to_unsigned(16#00000010#, ABITS)),
    wdata => x"DEADBEEF"
);
report "=== P1 DONE ===" severity note;

-- Test P2: Processor single READ
-- Drive: proc_addr=0x20, proc_oen=0
-- See:   psel=1, penable=1, pwrite=0
--        proc_rdata = 0x20FACADE (from APB slave model)
report "=== P2: Processor READ ===" severity note;
do_proc_read(
    addr => std_logic_vector(to_unsigned(16#00000020#, ABITS))
);
report "=== P2 DONE: expect proc_rdata=0x20FACADE ===" severity note;

-- Test P3: Back-to-back Processor Writes

```

```

-- Drive: 3 consecutive writes, no gap
-- See:  psel/penable toggling each cycle, pwwdata=0x11,0x22,0x33
report "=== P3: Back-to-back Processor Writes ===" severity note;
do_proc_write(
    std_logic_vector(to_unsigned(16#0000100#, ABITS)), x"11111111");
do_proc_write(
    std_logic_vector(to_unsigned(16#0000104#, ABITS)), x"22222222");
do_proc_write(
    std_logic_vector(to_unsigned(16#0000108#, ABITS)), x"33333333");
report "=== P3 DONE ===" severity note;

-- =====
-- AHB BURST TESTS (AHB Bridge → SRAM)
-- =====

-- — Test B1: SINGLE Write —————
-- 1 beat, hburst=000
-- Drive: haddr=0x40000040, hwwdata=0xCAFEBAABE
-- See:  sram_addr=0x040, sram_wdata=0xCAFEBAABE, sram_wen=0
--       hready=1 after 1 beat
report "=== B1: AHB SINGLE WRITE ===" severity note;
do_ahb_single(x"40000040", x"CAFEBAABE", '1');
report "=== B1 DONE ===" severity note;

-- — Test B2: SINGLE Read —————
-- Drive: haddr=0x40000080, hwwrite=0
-- See:  sram_oen=0, hrdata=0x80ABCDEF
report "=== B2: AHB SINGLE READ ===" severity note;
do_ahb_single(x"40000080", x"00000000", '0');
report "=== B2 DONE: expect hrdata=0x80ABCDEF ===" severity note;

-- — Test B3: INCR Write (undefined length, 4 beats) —————
-- hburst=001, 4 beats, addr increments by 4 each beat
-- Drive: haddr=0x40000100, beats=4
-- See:  sram_addr: 0x100,0x104,0x108,0x10C
--       sram_wdata: 0xAA..0xDD each beat
--       sram_wen=0 throughout
-- From table: allbrst=0, INCR → Incrementing burst
--             with BUSY cycles inserted if needed
report "=== B3: AHB INCR WRITE (4 beats) ===" severity note;
do_ahb_burst(x"40000100", INCR, '1');
report "=== B3 DONE ===" severity note;

-- — Test B4: INCR Read (undefined length, 4 beats) —————
-- See:  sram_oen=0, hrdata changes each beat
--       hrdata[beat0]=0x00ABCDEF, [beat1]=0x04ABCDEF etc

```

```

report "=== B4: AHB INCR READ (4 beats) ===" severity note;
do_ahb_burst(x"40000200", INCR, '0');
report "=== B4 DONE ===" severity note;

-- — Test B5: INCR4 Write (fixed 4 beats) —————
-- hburst=011, exactly 4 beats, addr +4 each beat
-- Drive: haddr=0x40000300
-- See:   sram_addr: 0x300,0x304,0x308,0x30C
--       4 sram_wen pulses
-- From table: allbrst=0, INCR4 → Fixed length 4 beats
report "=== B5: AHB INCR4 WRITE ===" severity note;
do_ahb_burst(x"40000300", INCR4, '1');
report "=== B5 DONE ===" severity note;

-- — Test B6: INCR4 Read —————
-- See:   4 hrdta values, address tagged
--       0x00ABCDEF, 0x04ABCDEF, 0x08ABCDEF, 0x0CABCDEF
report "=== B6: AHB INCR4 READ ===" severity note;
do_ahb_burst(x"40000400", INCR4, '0');
report "=== B6 DONE ===" severity note;

-- — Test B7: WRAP4 Write —————
-- hburst=010, 4 beats, address wraps within 16-byte boundary
-- Start at 0x40000418 (offset 8 within 16-byte block)
-- Beat addresses: 0x418, 0x41C, 0x410, 0x414 ← wraps!
-- From table: allbrst=0, WRAP4 → Malfunction/Not supported
--       allbrst=1 → Same burst type with BUSY cycles
-- We test to observe bridge behavior
report "=== B7: AHB WRAP4 WRITE ===" severity note;
do_ahb_burst(x"40000418", WRAP4, '1');
report "=== B7 DONE: observe addr wrap on waveform ===" severity note;

-- — Test B8: WRAP4 Read —————
-- Same wrapping but read direction
-- See address wrap: 0x418 → 0x41C → 0x410 → 0x414
report "=== B8: AHB WRAP4 READ ===" severity note;
do_ahb_burst(x"40000418", WRAP4, '0');
report "=== B8 DONE ===" severity note;

-- — Test B9: INCR8 Write —————
-- hburst=101, exactly 8 beats, addr +4 each beat
-- Drive: haddr=0x40000500
-- See:   8 consecutive sram_addr values, 8 wdata beats
report "=== B9: AHB INCR8 WRITE ===" severity note;
do_ahb_burst(x"40000500", INCR8, '1');
report "=== B9 DONE ===" severity note;

```

```
-- — Test B10: INCR8 Read —————
report "=== B10: AHB INCR8 READ ===" severity note;
do_ahb_burst(x"40000600", INCR8, '0');
report "=== B10 DONE ===" severity note;

-- — Test B11: WRAP8 Write —————
-- hburst=100, 8 beats, wraps within 32-byte boundary
-- Start at 0x40000720 (offset 0x20 = 32, start of boundary)
-- Addresses: 0x720,0x724,0x728,0x72C,0x730,0x734,0x738,0x73C
report "=== B11: AHB WRAP8 WRITE ===" severity note;
do_ahb_burst(x"40000720", WRAP8, '1');
report "=== B11 DONE ===" severity note;

-- — Test B12: WRAP8 Read —————
report "=== B12: AHB WRAP8 READ ===" severity note;
do_ahb_burst(x"40000720", WRAP8, '0');
report "=== B12 DONE ===" severity note;

-- — Test B13: INCR16 Write —————
-- hburst=111, exactly 16 beats, addr +4 each beat
-- Drive: haddr=0x40000800
-- See: 16 consecutive SRAM addresses 0x800..0x83C
report "=== B13: AHB INCR16 WRITE ===" severity note;
do_ahb_burst(x"40000800", INCR16, '1');
report "=== B13 DONE ===" severity note;

-- — Test B14: INCR16 Read —————
report "=== B14: AHB INCR16 READ ===" severity note;
do_ahb_burst(x"40000900", INCR16, '0');
report "=== B14 DONE ===" severity note;

-- — Test B15: WRAP16 Write —————
-- hburst=110, 16 beats, wraps within 64-byte boundary
-- Start at 0x40000A40 (start of 64-byte boundary)
report "=== B15: AHB WRAP16 WRITE ===" severity note;
do_ahb_burst(x"40000A40", WRAP16, '1');
report "=== B15 DONE ===" severity note;

-- — Test B16: WRAP16 Read —————
report "=== B16: AHB WRAP16 READ ===" severity note;
do_ahb_burst(x"40000A40", WRAP16, '0');
report "=== B16 DONE ===" severity note;

-- — Test B17: INCR4 Write with BUSY after beat 1 —————
-- Master inserts BUSY between beat 1 and beat 2
```

```
-- From table: bridge must handle BUSY and wait
-- See: htrans goes 10→11→01→01→11→11 (NONSEQ,SEQ,BUSY,BUSY,SEQ,SEQ)
--      bridge holds sram_ce1 low during BUSY
report "=== B17: AHB INCR4 WRITE with BUSY ===" severity note;
do_ahb_burst(x"40000B00", INCR4, '1', busy => 1);
report "=== B17 DONE: observe BUSY insertion on waveform ===" severity note;

-- — Test B18: INCR8 Read with BUSY after beat 3 —————
-- See: htrans inserts BUSY mid-burst
--      hrdata should still be valid when BUSY exits
report "=== B18: AHB INCR8 READ with BUSY ===" severity note;
do_ahb_burst(x"40000C00", INCR8, '0', busy => 3);
report "=== B18 DONE ===" severity note;

-- =====
-- END
-- =====

wait for 20 * CLK_PERIOD;
report "=== ALL TESTS COMPLETE ===" severity note;
report "Simulation finished" severity failure;

end process stimulus;

end architecture sim;
```

What You'll See Per Test

| Test            | Watch These Signals    | Expected Value                   |
|-----------------|------------------------|----------------------------------|
| P1 Write        | psel, penable, pwidth  | psel↑ penable↑ pwidth=0xDEADBEEF |
| P2 Read         | proc_rdata             | 0x20FACADE                       |
| P3 Back-to-back | psel/penable toggling  | pwidth=0x11,0x22,0x33            |
| B1 Single Write | sram_wdata, sram_wen   | 0xCAFEBABE, wen↓                 |
| B2 Single Read  | hrdata                 | 0x80ABCDEF                       |
| B3/B4 INCR      | sram_addr incrementing | +4 each beat                     |
| B5/B6 INCR4     | sram_addr × 4 beats    | exactly 4 pulses                 |
| B7/B8 WRAP4     | sram_addr wraps        | 0x418→0x41C→0x410→0x414          |

| Test           | Watch These Signals                       | Expected Value                            |
|----------------|-------------------------------------------|-------------------------------------------|
| B9/B10 INCR8   | <code>sram_addr</code> × 8 beats          | exactly 8 pulses                          |
| B11/B12 WRAP8  | <code>sram_addr</code> wraps at 32B       | wrap after 8 beats                        |
| B13/B14 INCR16 | <code>sram_addr</code> × 16 beats         | exactly 16 pulses                         |
| B15/B16 WRAP16 | <code>sram_addr</code> wraps at 64B       | wrap after 16 beats                       |
| B17 BUSY Write | <code>htrans</code>                       | <code>NONSEQ→SEQ→BUSY→BUSY→SEQ→SEQ</code> |
| B18 BUSY Read  | <code>htrans</code> + <code>hrdata</code> | hrdata valid after BUSY clears            |

